

TradeSaath Arena — Intern Onboarding Guide

A focused guide for an intern whose main work is the Arena module specifically. Read the general TradeSaath onboarding guide first for platform-wide context (stack, architecture, workflow rules) — this document goes deep on Arena itself.

Part 1: What Arena Is and Why It Exists

The pitch, in one line: Arena is "Duolingo for traders" — a gamified practice module bolted onto TradeSaath, built to make skill-building feel like a game instead of homework.

Why it exists (the product reasoning an intern should internalize): TradeSaath's core product analyzes a trader's *real* trading history after the fact — it's reflective, looking backward at trades already made. Arena is the opposite: it's a **practice environment** where users build pattern-recognition and decision-making skill *before* risking real money, through short repeatable game-like exercises. It's currently spec'd as a **standalone engagement module** — meaning it can be built and shipped on its own — with a longer-term plan to plug it into TradeSaath's personalized learning paths (once the main platform's AI has identified a user's psychological archetype/patterns, Arena could serve that specific user different puzzles suited to their weak spots, rather than the same content for everyone).

Currently spec'd components (this is the full known scope as of the last spec):

- **Chart Guess** — a game mode, most likely showing a partial price chart and asking the user to predict what happens next (this pattern is common in trading-education games; confirm the exact mechanic against the actual spec doc before building).
- **Pattern Spot** — a mode focused on recognizing specific chart/candlestick patterns.
- **Flash Quiz** — fast-paced quiz-style questions, likely testing trading knowledge/concepts under time pressure.
- **Weekly Leagues** — a recurring competitive structure where users are grouped and ranked against each other on a weekly cycle (this is a very Duolingo-specific mechanic — Duolingo's own leagues promote/demote users between tiers weekly, which is likely the reference model here).
- **Skill Tree** — a structured progression path, unlocking content/levels as a user demonstrates competence, rather than everything being available at once.
- **XP/streaks** — experience points for completing activities, and a "don't break the chain" daily streak mechanic — the single most important retention lever in apps like Duolingo.

- **"Saathi" mascot** — a branded character/mascot tying the gamified experience back to TradeSaath's identity (Saathi means "companion/partner" in Hindi — fitting for a coaching-companion brand).

Important scoping note for the intern: the above is the spec as currently on file. Before writing code, the intern's first task should be to **get the actual detailed spec/PRD document** for Arena (if one exists beyond this summary) — game mechanics like scoring formulas, exact XP curves, and league promotion/demotion rules need to be nailed down precisely; guessing at these numbers will cause rework.

Part 2: How Arena Fits Into the Existing Stack

Arena is not a separate product (unlike the standalone Tradiko-style prediction game project) — it's a module *within* TradeSaath, so it inherits the main platform's tech choices rather than picking its own:

Layer	What Arena uses	Why (recap from the platform-wide stack)
Frontend/backend	Next.js (App Router) , hosted on Vercel	Same app, same deploy pipeline — Arena is new pages/routes inside the existing project, not a new codebase
User accounts	Clerk	Arena users are the same TradeSaath users — no separate login system
Data storage	Supabase (Postgres)	XP totals, streak counts, league standings, skill-tree progress, and quiz/game results all need structured, relational storage — same reasoning as the rest of the platform (see main onboarding doc)
AI (if used)	Claude API / Gemini	Only if a game mode needs AI judgment (e.g., generating novel quiz questions, evaluating a free-text answer) — for anything with a clear right/wrong answer (like "did the price go up or down"), plain deterministic code should be used instead, per the platform's core cost-control principle

The one core lesson to carry over from the main platform into Arena work: *use AI only where judgment is genuinely needed; use plain rules-based code for anything mechanical.* Grading "was this chart-direction guess correct" is a deterministic comparison, not an AI task. Generating personalized commentary on *why* a guess pattern reveals a bias might

genuinely need AI. The intern should ask, for every feature: "does this need judgment, or just a comparison/calculation?"

Part 3: Data Modeling — What an Intern Should Think Through First

Before writing UI, Arena needs a few new database concepts (in Supabase) that don't exist elsewhere in TradeSaath yet. These are the areas to design carefully, since gamification systems are notoriously easy to get subtly wrong:

- **XP and streak tracking** — needs to answer: does XP ever decay? What exactly counts as "maintaining a streak" (once a day? a specific action?) What timezone is a "day" measured in for a global user base? (This matters *specifically* for TradeSaath since it's explicitly a global, not India-only, product — a naive "midnight IST" streak reset would be unfair to non-Indian users.)
 - **League standings** — needs a scheduled process (weekly) to snapshot rankings, promote/demote users between tiers, and reset the current period. This is a good candidate for a **scheduled/cron job** (a piece of code that runs automatically on a timer) rather than something calculated live on every page load.
 - **Skill tree progression** — needs a clear, explicit definition of prerequisites (what must be completed before the next node unlocks) stored as data, not hardcoded logic scattered through the UI — otherwise adding or reordering skill-tree content later becomes painful.
 - **RLS from day one** — exactly like the rest of TradeSaath, every Arena table needs Row Level Security so one user's XP/progress data can't be read or written to by another user. Given that a real RLS misconfiguration was already found and flagged as a PO launch blocker elsewhere in this codebase, this is the single most important thing to get right from the start in Arena's own tables too — not something to "add later."
-

Part 4: Suggested First-Two-Weeks Path for This Intern

1. **Day 1:** Read the general TradeSaath onboarding guide (stack, architecture, workflow rules) — Arena is built inside that same system, so none of that context is optional.
2. **Day 1-2:** Track down the actual Arena spec/PRD (beyond this summary) and read it fully before writing any code. If scoring formulas, XP curves, or league rules aren't fully specified yet, flag that early rather than guessing.
3. **Day 2-3:** Sketch the data model for XP, streaks, leagues, and skill-tree progress on paper/a doc first, and get it reviewed, before creating any Supabase tables — schema mistakes are expensive to fix once real user data is in them.

4. **Day 3-5:** Build the smallest possible end-to-end slice of **one** game mode (recommend starting with Flash Quiz — likely the simplest mechanically) fully working: play → score → XP awarded → saved to Supabase → shown back to the user. Get this one full loop working before touching a second mode.
 5. **Week 2:** Add streak tracking to that one mode, including RLS on the new tables from the start.
 6. **Ongoing:** Follow the same workflow rules as the rest of TradeSaath — `(npx tsc --noEmit)` before every commit, one focused commit per task, diffs reviewed before push, never touch the protected config files without checking first.
-

Open Questions to Resolve Before/During Build (Flag These to the Founder)

- Exact scoring/XP formulas for each game mode.
- League promotion/demotion rules and group sizes (how many users per league tier?).
- Timezone handling for streaks, given the global user base.
- Whether any game mode needs live/real-time multiplayer elements (this would introduce new infra needs — e.g., WebSockets — not otherwise part of the current stack) or if everything is asynchronous (each user plays independently, leaderboards update periodically).
- How Saathi the mascot is meant to appear technically — static illustrations, an animated character, or AI-generated dialogue tied into the coaching system.